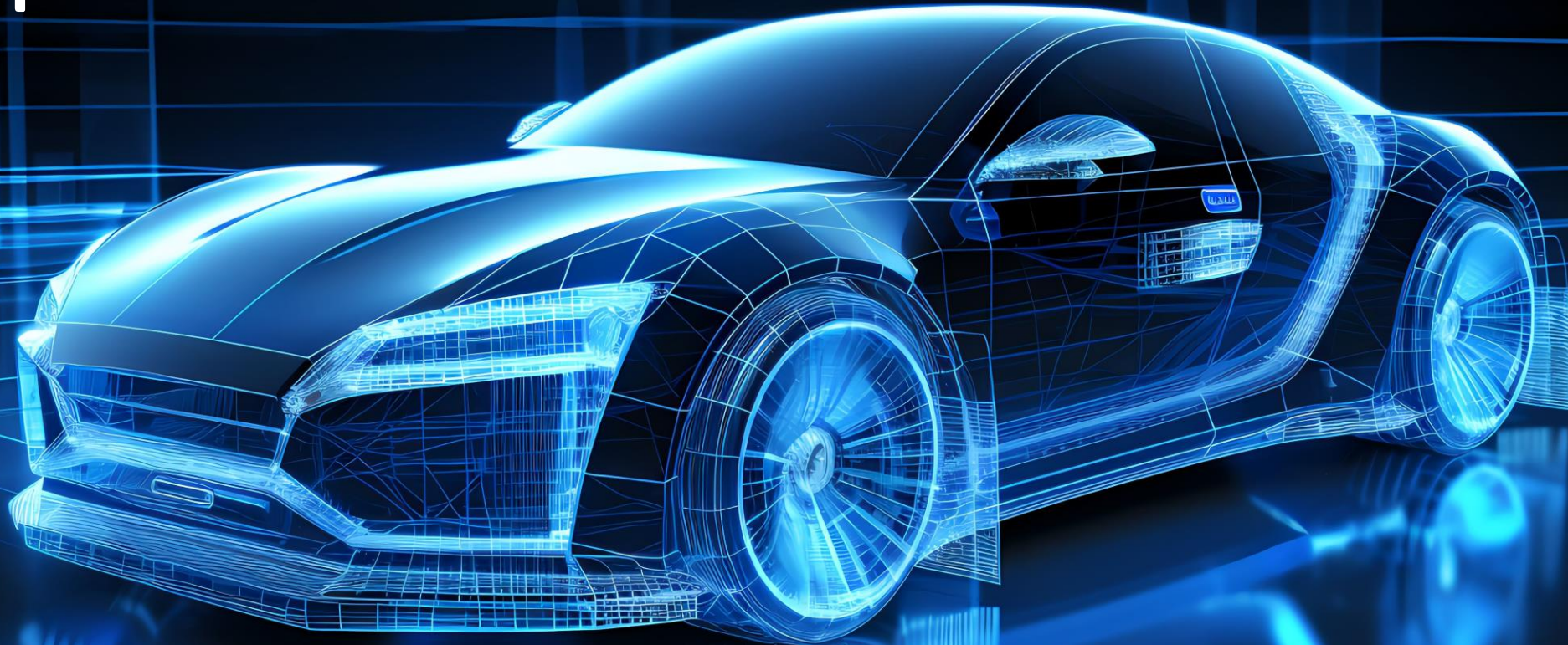


AUTOSYNC Infotainment System



Olamide Aderogba
G00401971

Research

Research

•Idea Development

- Idea generated based off the BMW (Idrive)
- Began as Spotify web page (Presented in proposal)
- Pivoted to car infotainment system
- Discovering key implementations into project

•Components/Tools

- HW: Laptop (Mac), webcam,
- SW: React, Node.js, Python, MediaPipe, Google Maps API, OpenAI API
- Shifted from Spotify API, ESP32-S3 Skainet to local music, OpenAI

•Research & Synthesis

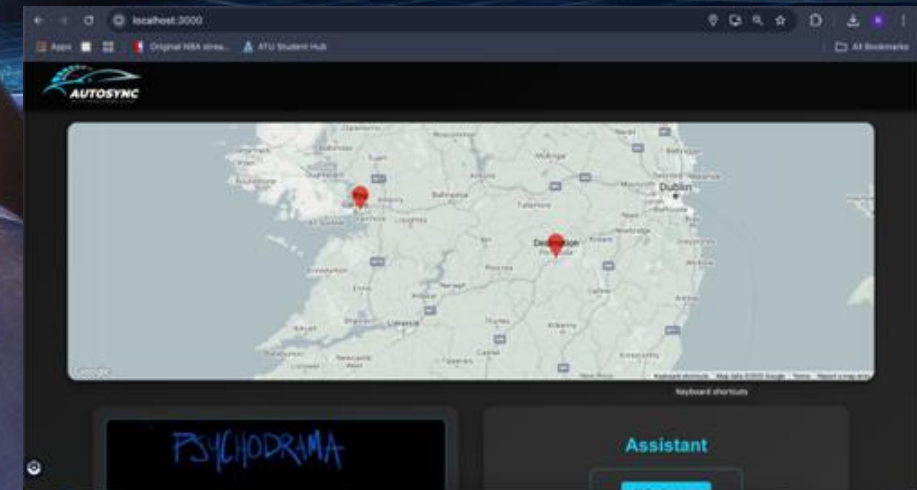
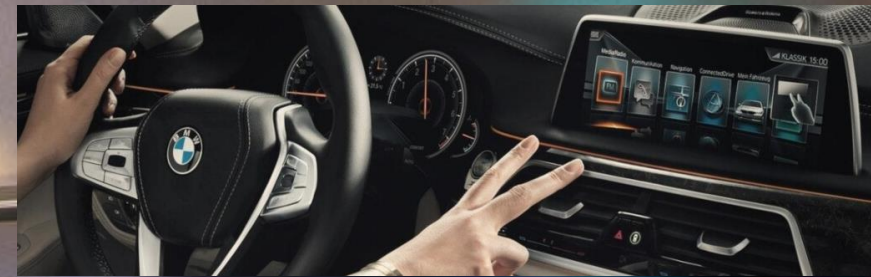
- Analysis within car – Unique idea & approach
- NHTSA/RSA stats for hands-free focus
- APIs (Spotify, Google Maps, OpenAI), gesture methods (TensorFlow, MediaPipe)
- Sources: API docs, papers [5], [6]
- BMV Gesture Control within cars

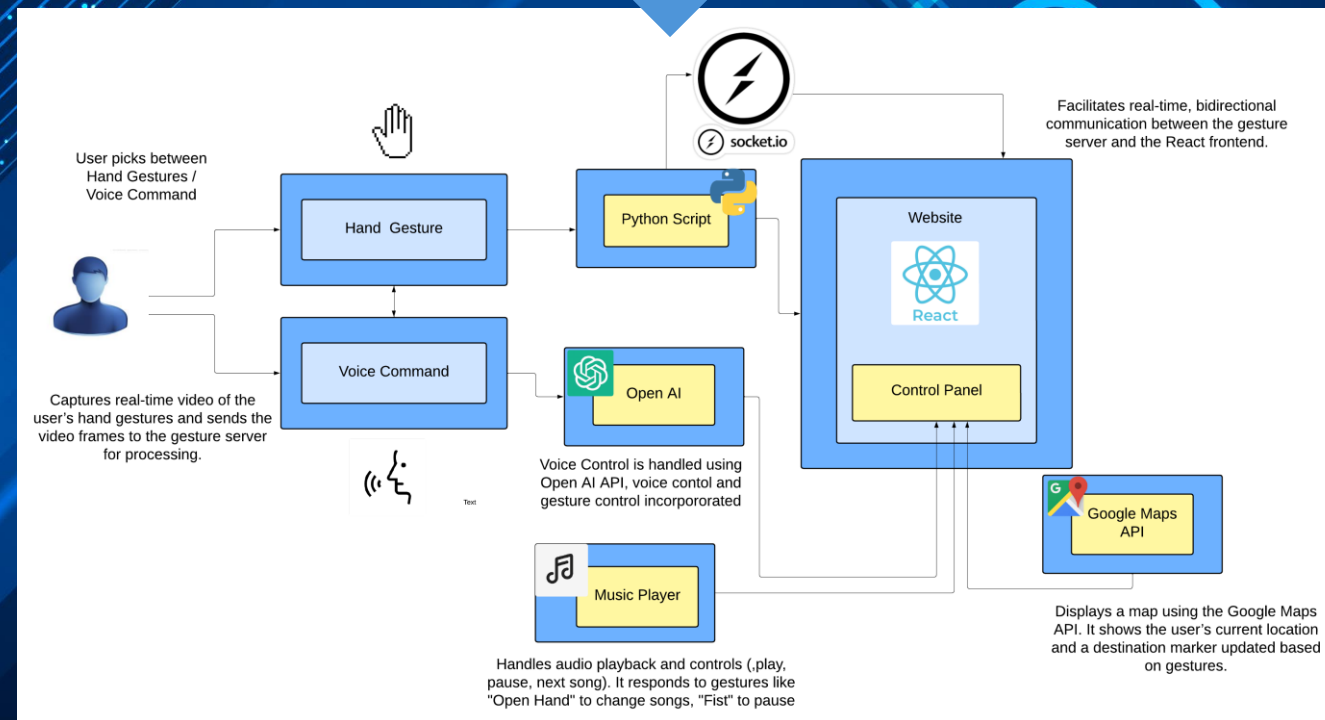
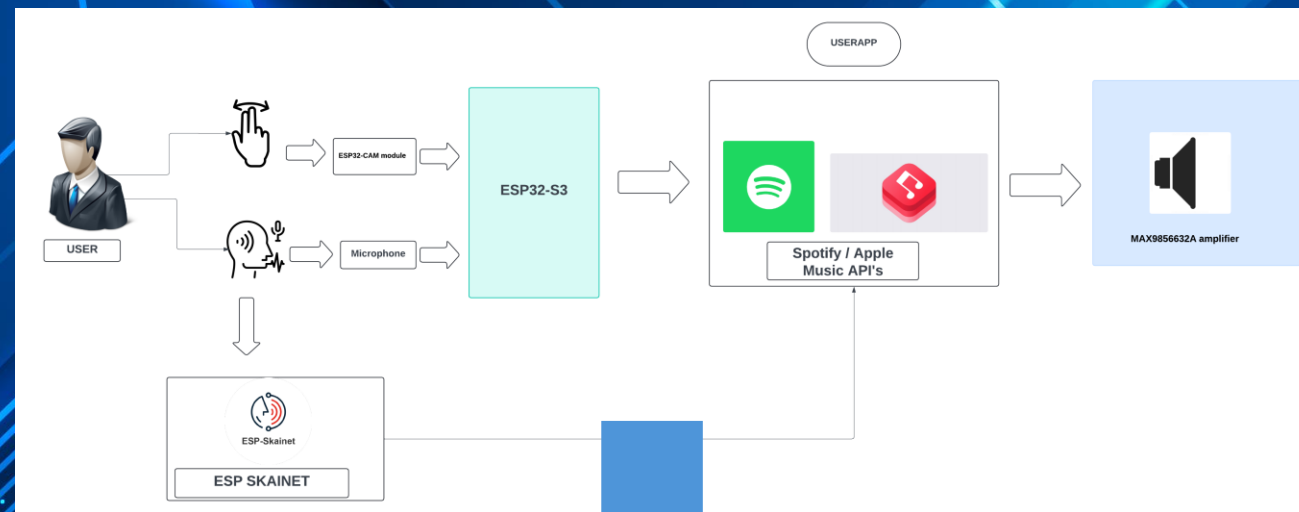
•Challenges

- Spotify URI delays (Sprints 1–2); OAuth fix
- ESP32-S3 setup delays (Sprint 3); dropped
- TensorFlow Object detection issues (Sprint 7); used MediaPipe

•References

- [1] Google Mediapipe 2025
- [5] Kim & Park, 2023
- [3] Socket.IO, 2025





Academic/ Technical Content

Complex Technical Development

- Built hands-free car infotainment system using gesture/voice control
- Integrated React frontend, Node.js backend, Python gesture recognition

Problem-Solving & Independence

- Resolved Spotify API ban by pivoting to local music
- Fixed TensorFlow issue by switching to MediaPipe
- Worked independently, with guidance from supervisor and advice from peers.

Unique Code & Algorithms

- Developed custom gesture recognition algorithm with MediaPipe
- Created AllInterface for voice command processing with OpenAI API
- Implemented real-time Socket.IO integration for gesture updates

Software Design & Analysis

- Designed React components (GoogleMapComponent, MusicPlayer)
- Analysed RSA/NHTSA stats to prioritise hands-free safety features
- Evaluated gesture accuracy (custom dataset vs. MediaPipe)

External Technologies & Third-Party Use

- Used Web Speech API for voice input
- Integrated Google Maps API, OpenAI API with proper refs [2], [3]
- Avoided overusing tutorials, built unique logic for gesture detection

Plagiarism Awareness

- Properly cited and referenced third-party APIs/ libraries and information within reports



Demonstration of Understanding

Overall Project Idea & Purpose

- Hands-free car infotainment to reduce driver distraction
- Integrated gesture/voice control for navigation, music

Implementation Overview

- React frontend for UI, Node.js backend for voice processing
- Python script for gesture recognition, linked via Socket.IO

Field Insights

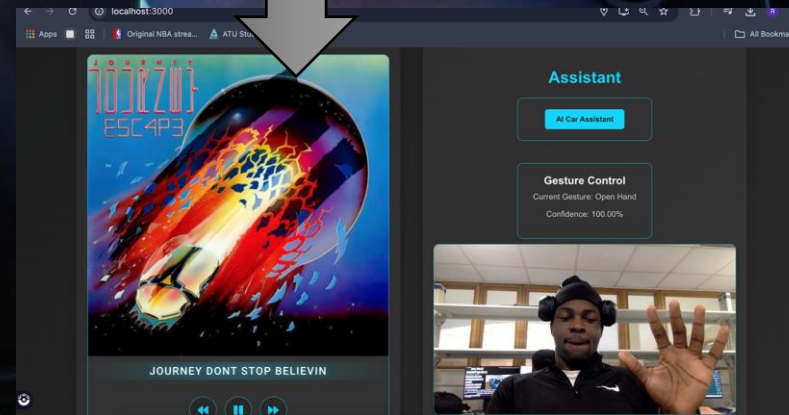
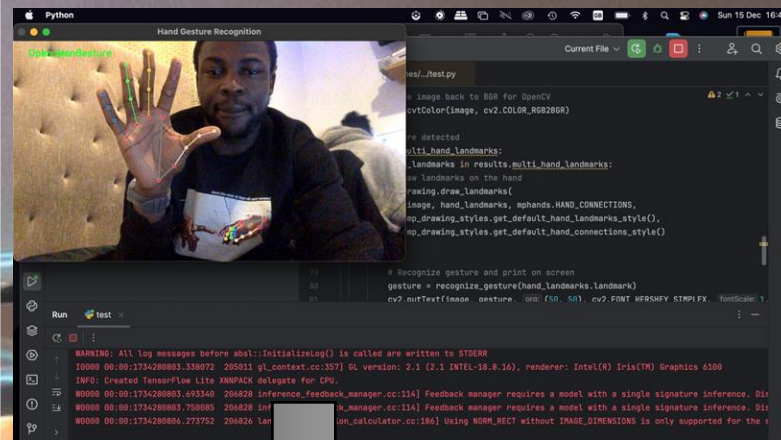
- In-vehicle tech: Hands-free aligns with safety trends
- Gesture/voice control: MediaPipe, OpenAI API benefits

Deep Learning

- Mastered MediaPipe for precise gesture detection
- Learned Web Speech API for robust voice input
- Understood Socket.IO for real-time system sync

Code Reuse & Acknowledgment

- Reused Socket.IO [2], MediaPipe examples [3], cited properly



Organisation

Project Organisation Skills

- Structured 24 weeks into Jira sprints
- Managed solo: Planned, coded, tested, documented
- Attended Stand up meetings, updated OneNote log weekly

Task Planning & Reflection

- Prioritise major tasks then pivot tasks
- Weekly reflections: Adjusted plans post-API ban

Plan Documentation

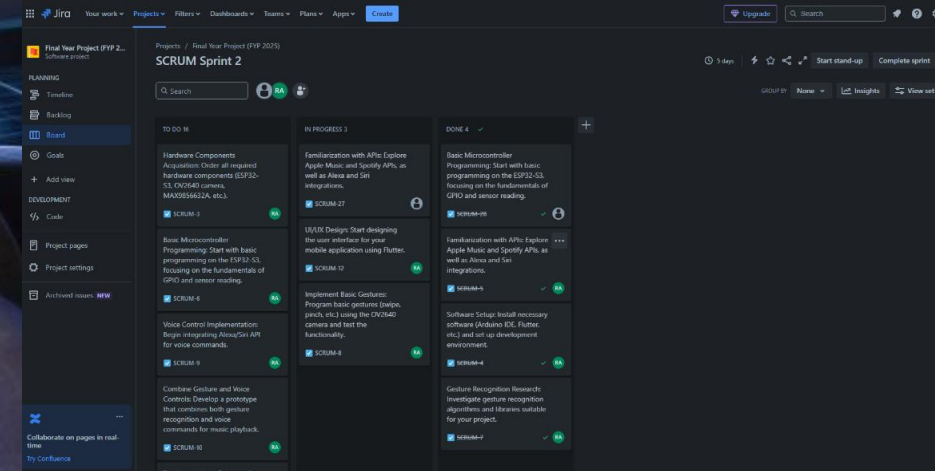
- Updated Jira: Added pivot tasks
- Documented timeline

Commits to Github

- Committed code to GitHub repository

Demo Preparation

- Prepared demo: Tested gestures, voice commands



What I doing this week?

This week I plan to shift the concept for my React web page from one with a music interface to that of an infotainment car system. The removal of the Spotify API, upon which I had planned to develop the majority of my project, is the reason behind this shift. Instead of focusing on music streaming, I now plan to develop a more holistic infotainment system that has the capability to support various types of media and services. For further enhancing the richness of the user experience, I plan to incorporate the Google APIs, like services such as Google Maps for navigation, Google Assistant for voice commands, and potentially YouTube for watch video or other entertainment functions.

This shift will require redesigning the user interface to support multiple functionalities such as navigation, media control, and voice interaction, while also ensuring a responsive, easy-to-use design suitable for an in-car environment. It is also working on embedding the Google API into my project, which will involve setting up API keys, configuring the necessary endpoints, and handling authentication securely. In total, the week will be used to close the scope of the project to Google's product suite and design an infotainment system that meets the needs of an interactive and multifunctional in-car experience.

Blockers?

One of the blockers I'm facing is issues building the ESP-Skainet repository on my ESP32-S3. The problem seems to be version compatibility among the ESP-Skainet repo, the ESP32-S3 hardware, and the development environment. Despite following the setup instructions carefully, I've encountered build errors, which means there are probably version mismatches among the ESP-IDF framework, ESP-Skainet libraries, or the toolchain being used. This has stopped development of adding voice recognition features to the project, as the necessary libraries are not compiling correctly.

To resolve this, I'm going to research and verify the versions of dependencies required and ESP32-S3 and ESP-Skainet repo compatible. I'll also review the issues tab of the repository to determine if others have encountered this problem and if there are suggested solutions or workarounds. With this issue resolved, I'll be able to move forward with voice recognition integration and continue with the project further.

Another issue I'm currently having is resolving the version compatibility of the TensorFlow Object Detection API which is not allowing me to progress much in the area of the hand gesture detection.

Initiative And Engagement

Proactive Approach

- Self initiated pivot after Spotify API ban
- Explored TensorFlow for gestures before MediaPipe switch
- Approached my project to be unique as possible – proactively looking for areas to improve

Self-Motivation

- Solo project: Managed 24 weeks, Jira sprints
- Consistently met deadlines despite challenges and other module deadlines

Seeking Feedback

- Consulted supervisor on pivot strategy
- Incorporated peer feedback on different aspect of project and also on report review

Engagement

- Presented progress at review sessions
- Engaged users during Apr. 2025 demo for feedback
- Stand up Meetings
- General checkup with peers during project class



References

- [1] Google, "Google Maps Platform Documentation," Google Cloud, 2025. [Online]. Available: <https://developers.google.com/maps/documentation>. [Accessed: Apr. 12, 2025].
- [2] OpenAI, "OpenAI API Documentation," OpenAI, 2025. [Online]. Available: <https://platform.openai.com/docs/api-reference>. [Accessed: Apr. 12, 2025].
- [3] Google, "MediaPipe Hands Documentation," MediaPipe, 2025. [Online]. Available: https://developers.google.com/mediapipe/solutions/vision/hand_landmarker. [Accessed: Apr. 12, 2025].
- [4] React, "React Documentation," React, 2025. [Online]. Available: <https://react.dev/reference>. [Accessed: Apr. 14, 2025].
- [5] Socket.IO, "Socket.IO Documentation," Socket.IO, 2025. [Online]. Available: <https://socket.io/docs/v4/>. [Accessed: Apr. 15, 2025].
- [6] OpenCV Team, "OpenCV: Real-Time Computer Vision with OpenCV," OpenCV Documentation, 2025. [Online]. Available: <https://docs.opencv.org/4.x/>. [Accessed: Apr. 25, 2025].